

LAB 2 · 7세그먼트 자리 스캔

VS Code에서 직접 작성·검증한 다음 장비로 확인한다.

작성일 2026. 09. 10. · 이해리

Vivado 실습

목차

프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code 시뮬레이션과 수정

구현과 장치 기록

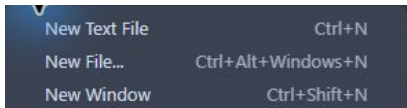
레포트 공통 예습 전체 목차

v2.0.1 템플릿을 새 이름으로 clone

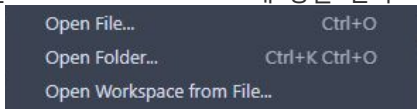
Windows PowerShell에서 한 줄씩 실행한다.

```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_08_segment_scan
cd lab2_08_segment_scan
git switch -c main
code LAB1.code-workspace
```

File → New Window



Ctrl+Shift+N 또는 File → New Window. 새 창을 연다.



File → Open Workspace from File...을 누른다.

workspace와 추천 확장

1. lab2_08_segment_scan 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src·sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/segment_scan8.v
src	src/input_frontend.v
src	src/lab2_segment_scan.v
sim	sim/tb_segment_scan8.sv
constraints	constraints/lab2_segment_scan.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_segment_scan8.sv`

`simulation_top`: `tb_segment_scan8`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.

```
1  {  
2  "sources": [  
3    "src/segment_scan8.v",  
4    "src/input_frontend.v",  
5    "src/lab2_segment_scan.v"  
6  ],  
7  "testbench": "sim/tb_segment_scan8.sv",  
8  "simulation_top": "tb_segment_scan8"  
9  }
```

이번 회로의 관찰 방법

SW1..4(sw[7:4]): 첫 자리 숫자. COM[7]부터 입력값, 1,2,3,4,5,6,7 순서. 1 kHz에서 자리당 62.5 Hz, blanking 포함.

K4로 초기화한 뒤 SW1~4로 첫 자리 숫자를 바꾼다. 스캔은 자동으로 진행되며 N8 버튼 입력은 사용하지 않는다.

여덟 자리를 시간으로 나누어 켜다

digits는 8개의 4비트 숫자를 묶은 32비트 입력이다. index=0은 가장 낮은 4비트, index=7은 가장 높은 4비트를 선택한다.

index는 3비트라 7 다음 증가에서 0으로 순환한다. nibble = digits » (index * 4)의 결과를 4비트에 대입하여 해당 숫자를 얻는다.

select는 논리적인 active-high 자리 선택이다. 실제 COM 핀의 순서와 극성은 보드 모듈에서 변환한다.

켜기와 blank 구간

이전 blank	enable	에지 뒤 동작
1	1	같은 index 활성화, blank=0
0	1	다음 index 준비, blank=1
무관	0	index와 blank 유지

리셋은 index=0, blank=1로 시작한다. blank 동안 select=00이라 모든 자리가 꺼지고 다음 숫자의 세그먼트 값이 준비된다.

1 kHz에서 자리당 62.5 Hz

보드 모듈은 enable을 항상 1로 연결한다. 한 자리의 활성 1 ms와 blank 1 ms를 합쳐 2 ms다.

8자리 순회는 16 ms이므로 각 자리의 반복 주파수는 $1 / 0.016 = 62.5 \text{ Hz}$ 다. 각 자리는 16 ms 중 1 ms 켜진다.

TB는 유지 조건도 검사하기 위해 enable을 중간에 0으로 내린다. TB의 파형 길이를 실제 보드 스캔 주기로 그대로 해석하지 않는다.

숫자와 물리 핀의 순서

segments[7:0]은 a,b,c,d,e,f,g,dp 순서다. 예를 들어 0은 FC, 1은 60이며 dp는 0으로 꺼져 있다.

보드의 seg_data는 segments를 그대로 전달한다. seg_com은 자리 순서를 뒤집고 반전한다.

index=0이면 COM[7]만 0이라 seg_com=7F다. 다음 자리는 BF, 그다음 DF다. blank에서는 FF로 모두 꺼진다.

segment_scan8.v · 1-16행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Logical outputs are active-high. Apply board polarity in the wrapper.
3 | // Each slot contains one active interval followed by one blank interval.
4 | module segment_scan8 (
5 |     input wire clk, rst, enable,
6 |     input wire [31:0] digits,
7 |     output reg [7:0] select,
8 |     output reg [7:0] segments,
9 |     output reg [2:0] index
10 | );
11 |     reg blank;
12 |     reg [3:0] nibble;
13 |     always @(posedge clk) begin
14 |         if (rst) begin
15 |             index <= 0;
16 |             blank <= 1'b1;
```

segment_scan8.v · 17-32행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
17         end else if (enable) begin
18             blank <= ~blank;
19             if (!blank) index <= index + 1'b1;
20         end
21     end
22     always @* begin
23         nibble = digits >> (index * 4);
24         select = blank ? 8'h00 : (8'h01 << index);
25         // abcdefg, dp; decimal point remains off.
26         case (nibble)
27             4'h0: segments = 8'b11111100;
28             4'h1: segments = 8'b01100000;
29             4'h2: segments = 8'b11011010;
30             4'h3: segments = 8'b11110010;
31             4'h4: segments = 8'b01100110;
32             4'h5: segments = 8'b10110110;
```

segment_scan8.v · 33–46행

src/segment_scan8.v를 열고 해당 구간을 직접 입력한다.

```
33      4'h6: segments = 8'b10111110;  
34      4'h7: segments = 8'b11100000;  
35      4'h8: segments = 8'b11111110;  
36      4'h9: segments = 8'b11110110;  
37      4'ha: segments = 8'b11101110;  
38      4'hb: segments = 8'b00111110;  
39      4'hc: segments = 8'b10011100;  
40      4'hd: segments = 8'b01111010;  
41      4'he: segments = 8'b10011110;  
42      4'hf: segments = 8'b10001110;  
43      default: segments = 8'h00;  
44  endcase  
45  end  
46 endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17  assign reset = reset_pipe[1];
18  assign switches = sw_sync;
19  // Asynchronous assertion; release follows two main-clock edges.
20  always @(posedge clk or posedge rst) begin
21      if (rst) reset_pipe <= 2'b11;
22      else reset_pipe <= {reset_pipe[0], 1'b0};
23  end
24  always @(posedge clk or posedge reset) begin
25      if (reset) begin
26          button_meta <= 0; button_sync <= 0;
27          sw_meta <= 0; sw_sync <= 0;
28      end else begin
29          button_meta <= button; button_sync <= button_meta;
30          sw_meta <= sw; sw_sync <= sw_meta;
31      end
32  end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_segment_scan.v · 1-16행

src/lab2_segment_scan.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3 | // K4: reset. N8: step button. DIPSW1..8 = sw[7]..sw[0].
4 | module lab2_segment_scan #(parameter integer STABLE_CYCLES = 20) (
5 |     input wire clk, rst, button,
6 |     input wire [7:0] sw,
7 |     output wire [7:0] led,
8 |     output wire [7:0] seg_data, seg_com
9 | );
10 |     wire reset, press;
11 |     wire [7:0] switches;
12 |     input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
13 |         clk, rst, button, sw, reset, press, switches);
14 |     wire [7:0] selected, segments;
15 |     wire [2:0] index;
16 |     wire [31:0] digits = {28'h7654321, switches[7:4]};
```

lab2_segment_scan.v · 17-23행

src/lab2_segment_scan.v를 열고 해당 구간을 직접 입력한다.

```
17     segment_scan8 core(clk, reset, 1'b1, digits, selected, segments, index);
18     // Legacy array selects are active-low; index 0 is COM[7].
19     assign seg_com = ~{selected[0],selected[1],selected[2],selected[3],
20                       selected[4],selected[5],selected[6],selected[7]};
21     assign seg_data = segments;
22     assign led = {5'b00000, index};
23 endmodule
```

TB는 0-F와 자리 순서를 검사한다

digits=76543210에서 0-7, digits=FEDCBA98에서 8-F를 읽는다. 각 묶음마다 여덟 자리를 두 바퀴 돈다.

한 자리에서 index, one-hot 선택, 숫자 패턴, 활성 유지, blank, blank 유지의 여섯 항목을 검사한다.

2묶음 × 2바퀴 × 8자리 × 6개 + 처음/마지막 리셋 2개 = 194개다. 전체 종료는 1306 ns다.

첫 두 자리의 파형을 읽는다

검사 시각	index	select	동작
6 ns	0	00	리셋 blank
16 ns	0	01	숫자 0 활성화, segments=FC
26 ns	0	01	enable=0 활성화 유지
36 ns	1	00	다음 자리 준비
46 ns	1	00	enable=0 blank 유지
56 ns	1	02	숫자 1 활성화, segments=60

blank에서도 segments는 다음 숫자를 나타낼 수 있다. 꺼짐 여부는 segments=0이 아니라 자리 선택 select=0으로 판단한다.

tb_segment_scan8.sv · 1-16행

sim/tb_segment_scan8.sv를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module tb_segment_scan8;
3  reg clk=0, rst=1;
4  integer checks=0;
5  always #5 clk=~clk;
6  task step; begin @(posedge clk); #1; end endtask
7  task check(input condition, input [8*100-1:0] description);
8  begin
9      checks=checks+1;
10     if (condition !== 1'b1) begin
11         $display("LAB2_FAIL %0s time=%0t",description,$time);
12         $fatal(1,"check failed");
13     end
14 end
15 endtask
16 task finish;
```

tb_segment_scan8.sv · 17-32행

sim/tb_segment_scan8.sv를 열고 해당 구간을 직접 입력한다.

```
17 begin
18     $display("LAB2_PASS segment_scan8 checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_segment_scan8); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 reg enable=0;
25 reg [31:0] digits=32'h76543210;
26 wire [7:0] select,segments;
27 wire [2:0] index;
28 segment_scan8 dut(clk,rst,enable,digits,select,segments,index);
29 reg [7:0] expected[0:15];
30 integer bank, position, lap;
31 initial begin
32     // Truth-table values, ordered 0..F with abcdefg and dp.
```

tb_segment_scan8.sv · 33-48행

sim/tb_segment_scan8.sv를 열고 해당 구간을 직접 입력한다.

```
33 expected[0]=8'hfc; expected[1]=8'h60; expected[2]=8'hda; expected[3]=8'hf2;
34 expected[4]=8'h66; expected[5]=8'hb6; expected[6]=8'hbe; expected[7]=8'he0;
35 expected[8]=8'hfe; expected[9]=8'hf6; expected[10]=8'hee; expected[11]=8'h3e;
36 expected[12]=8'h9c; expected[13]=8'h7a; expected[14]=8'h9e; expected[15]=8'h8e;
37 step; check(select===0 && index===0,"reset blanks digit zero"); rst=0;
38 for(bank=0;bank<2;bank=bank+1) begin
39     digits=bank ? 32'hfedcba98 : 32'h76543210;
40     for(lap=0;lap<2;lap=lap+1) begin
41         for(position=0;position<8;position=position+1) begin
42             enable=1; step;
43             check(index===position,"position order");
44             check(select==(1<<position),"exactly intended digit active");
45             check(segments===expected[bank*8+position],"hex truth table");
46             enable=0; step;
47             check(select==(1<<position),"enable zero holds active slot");
48             enable=1; step;
```

tb_segment_scan8.sv · 49-57행

sim/tb_segment_scan8.sv를 열고 해당 구간을 직접 입력한다.

```
49         check(select===0,"blank interval between every digit");
50         enable=0; step; check(select===0,"blank interval hold");
51     end
52 end
53 end
54 enable=1; step; rst=1; step;
55 check(select===0 && index===0,"reset from active scan"); finish;
56 end
57 endmodule
```

XDC를 직접 작성한다

1. constraints 폴더의 XDC를 열고 전체 내용을 입력한다.
2. get_ports의 이름과 비트 번호를 board top 선언과 대조한다.
3. 같은 핀을 서로 다른 포트에 중복 지정하지 않는다.
4. File → Save All. Icarus 기능 시뮬레이션은 XDC를 사용하지 않는다.

lab2_segment_scan.xdc · 1-16행

constraints/lab2_segment_scan.xdc를 열고 해당 구간을 직접 입력한다.

```
1 # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2 # Pin source and array polarity: evidence/board-pin-provenance.json.
3 set_property PACKAGE_PIN B6 [get_ports {clk}]
4 set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5 set_property PACKAGE_PIN K4 [get_ports {rst}]
6 set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7 set_property PACKAGE_PIN N8 [get_ports {button}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9 set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_segment_scan.xdc · 17-32행

constraints/lab2_segment_scan.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_segment_scan.xdc · 33–48행

constraints/lab2_segment_scan.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 set_property PACKAGE_PIN H2 [get_ports {seg_data[0]}]
42 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[0]}]
43 set_property PACKAGE_PIN J7 [get_ports {seg_data[1]}]
44 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[1]}]
45 set_property PACKAGE_PIN J3 [get_ports {seg_data[2]}]
46 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[2]}]
47 set_property PACKAGE_PIN J1 [get_ports {seg_data[3]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[3]}]
```

lab2_segment_scan.xdc · 49–64행

constraints/lab2_segment_scan.xdc를 열고 해당 구간을 직접 입력한다.

```
49 set_property PACKAGE_PIN E4 [get_ports {seg_data[4]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[4]}]
51 set_property PACKAGE_PIN E2 [get_ports {seg_data[5]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[5]}]
53 set_property PACKAGE_PIN F5 [get_ports {seg_data[6]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[6]}]
55 set_property PACKAGE_PIN F1 [get_ports {seg_data[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[7]}]
57 set_property PACKAGE_PIN K5 [get_ports {seg_com[0]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[0]}]
59 set_property PACKAGE_PIN K3 [get_ports {seg_com[1]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[1]}]
61 set_property PACKAGE_PIN K1 [get_ports {seg_com[2]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[2]}]
63 set_property PACKAGE_PIN L6 [get_ports {seg_com[3]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[3]}]
```

lab2_segment_scan.xdc • 65–74행

constraints/lab2_segment_scan.xdc를 열고 해당 구간을 직접 입력한다.

```
65  set_property PACKAGE_PIN G3 [get_ports {seg_com[4]}]
66  set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[4]}]
67  set_property PACKAGE_PIN G1 [get_ports {seg_com[5]}]
68  set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[5]}]
69  set_property PACKAGE_PIN H6 [get_ports {seg_com[6]}]
70  set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[6]}]
71  set_property PACKAGE_PIN H4 [get_ports {seg_com[7]}]
72  set_property IOSTANDARD LVCMOS33 [get_ports {seg_com[7]}]
73  create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
74  set_false_path -from [get_ports {rst button sw[*]}]
```

VS Code에서 먼저 시뮬레이션

1. File → Save All.
2. Terminal → Run Task... → 02 Simulate.
3. 첫 오류가 있으면 파일 경로·행 번호와 이전 행의 세미콜론부터 확인.
4. LAB2_PASS 또는 LAB2_INTEGRATED_PASS와 검사 수·종료 시각 확인.
5. Terminal → Run Task... → 03 Open waveform.

파형에서 확인할 신호

1. clk·rst·제어 입력·저장값·출력을 파형에 추가한다.
2. 상승 에지 직전 입력과 에지 뒤 상태를 비교한다.
3. 리셋 우선·enable=0 유지·마지막 검사 시각을 확인한다.
4. 출력만 본 화면과 전체 파형을 각각 저장한다.
5. 새 실행 로그와 파형인지 확인한다. 이전 PASS를 재사용하지 않는다.

정상 실행의 기준

```
LAB2_PASS segment_scan8 checks=194
```

Icarus 종료 시각: 1306ns. 로그가 ps로 표시되면 1,000으로 나눠 ns와 비교한다.

이 수치는 교안의 입력 순서·TB·파라미터에 대한 기준이다.

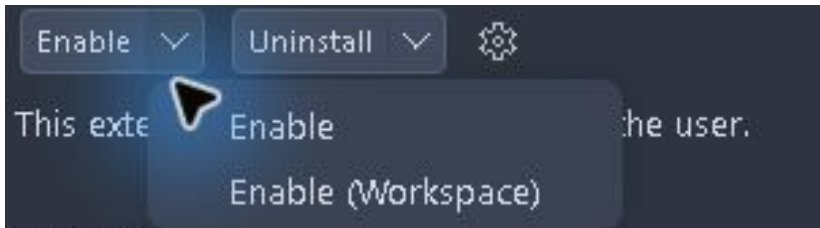
검사 수만 보지 말고 예상 상태와 실제 파형이 일치하는 이유를 설명한다.

7세그먼트 자기검사 통과

```
LAB2_PASS segment_scan8 checks=194
```

Ctrl+Shift+B 또는 Terminal → Run Task... → 02 Simulate. LAB2_PASS segment_scan8 checks=194를 확인한다. 종료 시각 1306000 ps는 1306 ns다.

파형 확장 활성화



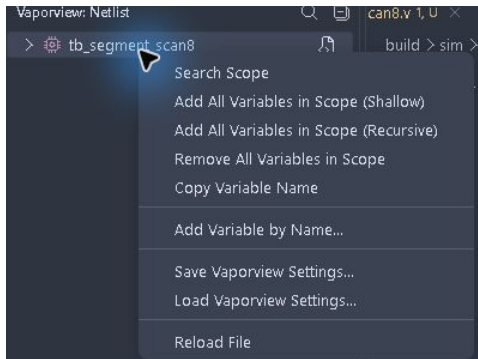
Extensions에서 @installed vaporview 검색. 꺼져 있으면 Enable 옆 화살표 → Enable (Workspace). 설치되어 있지 않으면 앞의 추천 확장 절차를 따른다.

파형 파일 열기



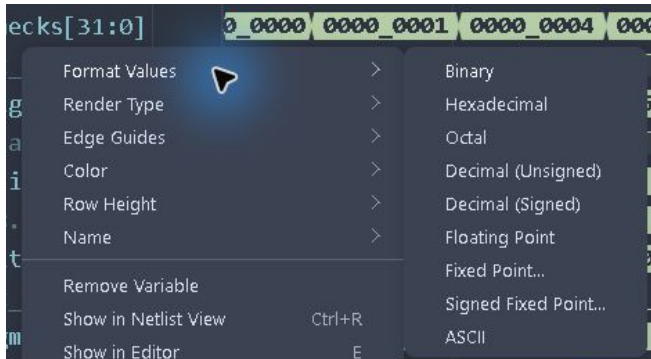
build/sim/wave.vcd를 연다. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView. 빈 파형 화면에서 Netlist View를 누른다.

테스트벤치 신호 추가



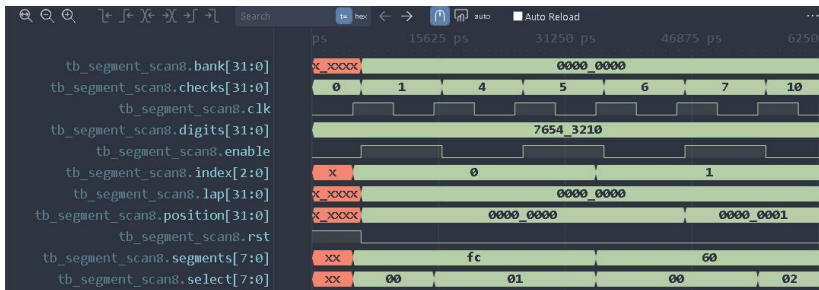
tb_segment_scan8 이름을 우클릭 → Add All Variables in Scope (Shallow). clk.rst.enable.digits.index.segments.select와 검사 변수가 추가된다.

검사 번호를 십진수로



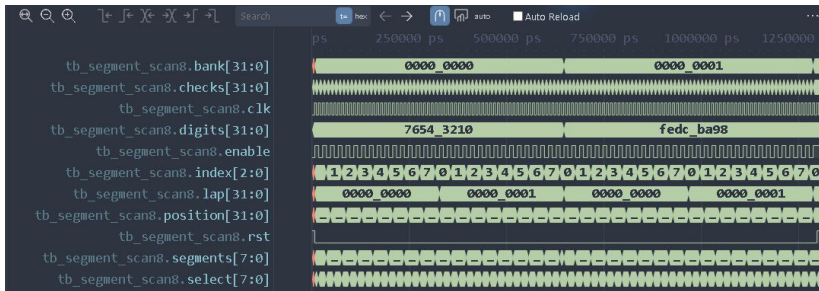
파형의 checks 이름을 우클릭 → Format Values → Decimal (Unsigned).
이름 열 경계를 오른쪽으로 끌어 신호 이름이 모두 보이게 한다.

자리 선택 사이의 소등 구간



단위는 ps다. 15000 ps = 15 ns. select는 15 ns에 01, 35 ns에 00, 55 ns에 02가 된다. 00인 구간은 모든 자리를 끄는 구간이다. enable=0인 25 ns와 45 ns 에지에서는 각각 점등·소등 상태가 유지된다.

두 입력 묶음과 네 번의 순회



Zoom Fit으로 전체 1306 ns를 본다. digits는 76543210에서 fedcba98로 바뀐다. 각 입력 묶음마다 index=0~7을 두 번 순회해 0~F의 세그먼트 패턴을 검사한다. 마지막에는 rst로 index=0·select=00을 확인한다.

파형과 실제 보드의 극성 연결

코어 select는 active-high다. 01은 index=0 자리 하나를 선택하고 00은 모든 자리를 끈다. segments는 abcdefg와 dp 순서이며 dp는 항상 0이다.

보드 연결 모듈은 select의 비트 순서를 뒤집고 반전하여 seg_com에 연결한다. 따라서 코어 index=0은 보드 COM[7]에 해당하며, 코어 select=00은 보드 seg_com=FF가 된다.

테스트벤치는 enable을 멈춰 유지 동작도 검사한다. 보드 모듈은 enable=1로 연결해 1 kHz 클록에서 자동 순환한다. N8 버튼을 누르는 동작과 자동 스캔을 혼동하지 않는다.

수정 실험 · 실패와 복구

1. 정상 로그와 파형을 먼저 보관한다.
2. 자리 전환 blanking 제거하기 TB 기대값은 유지한다.
3. Save All → 02 Simulate. LAB2_FAIL의 검사 이름·시각을 읽는다.
4. 어떤 입력과 이전 상태에서 불일치가 발생했는지 계산한다.
5. RTL을 복구·저장·재실행하고 전체 PASS를 확인한다.

실험 전 레포트

1. 동작 표·상태 전이·타이밍을 예상한다.
2. RTL·TB·XDC·설정의 역할을 설명한다.
3. 정상/변경/복구 3행 비교표와 실행 로그를 넣는다.
4. VS Code 파형을 확대해 입력·이전 상태·다음 상태를 설명한다.
5. 장비 클록·핀·예상 LED 동작을 정리한 뒤 Vivado로 진행한다.

스캔 파형·수정 실험 해설

`clk.rst.enable.digits.index.select.segments`를 추가한다. `index`는 정수, `select`와 `segments`는 16진수로 보면 표와 비교하기 쉽다.

`select` 식의 `blank` ? `8'h00` : 부분을 제거하면 6 ns의 `reset blanks digit zero` 검사가 먼저 실패한다. 기대 `select=00` 대신 `01`이 나온다.

이 변형은 자리 사이 `blank`뿐 아니라 리셋 `blank`도 없앤다. 첫 실패가 왜 리셋 검사인지 설명하고 원복 뒤 194개 `PASS·1306 ns` 종료를 확인한다.

Vivado에 같은 원본 연결

VS Code 정상·변경·원복 검사와 실험 전 레포트를 마친 뒤 시작한다.

설계 Top은 lab2_segment_scan, 시뮬레이션 Top은 tb_segment_scan8이다.
시뮬레이션은 코어를 검사하며 보드 연결 모듈은 실제 핀 극성을 연결한다.

프로젝트 이름과 위치

Project name: ×

Project location: × ...

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_08_segment_scan/vivado

File → Project → New → Next. 이름은 lab2_segment_scan, 위치는 자신의 lab2_08_segment_scan/vivado. Create project subdirectory 해제 → Next.

RTL 프로젝트 선택

- ☒ RTL Project
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL
- ☒ Do not specify sources at this time
- ☐ Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time을 선택한다. Next.

정확한 부품 선택

Parts	Boards	Q· xc7s75fgga484-1 (3 matches)			
Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs
xc7s75fgga484-1	484	338	48000	96000	90
xc7s75fgga484-1IL	484	338	48000	96000	90
xc7s75fgga484-1Q	484	338	48000	96000	90

Parts 검색란에 xc7s75fgga484-1을 입력한다. 같은 행 선택 → Next.
-1IL·-1Q와 구분한다.

생성 요약 확인

New Project Summary

- A new RTL project named 'lab2_segment_scan' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

lab2_segment_scan·Spartan-7·fgga484·-1을 확인하고 Finish. 이전 프로젝트가 열려 있으면 저장 상태를 확인한 뒤 닫는다.

원본 RTL 참조

	Index	Name	Library	Version	Location
●	1	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp
●	2	lab2_segment_scan.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp
●	3	segment_scan8.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp

☒ Scan and add RTL include files into project

☐ Copy sources into project

Add Sources → Add or create design sources → Next → Add Files. src의 input_frontend.v·lab2_segment_scan.v·segment_scan8.v 선택 → OK. 빈 design.v는 선택하지 않는다. Copy sources into project 해제 → Finish.

테스트벤치 등록

Specify simulation set: sim_1

+	-	↑	↓			
	Index	Name	Library	Version	Location	
●	1	tb_segment_scan8.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp	

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

Add Sources → Add or create simulation sources → Next → Add Files → sim/tb_segment_scan8.sv. sim_1과 Include all design sources 유지, Copy sources 해제 → Finish.

XDC 원본 참조

Specify constraint set: constrs_1 (active)

+ - ↑ ↓	
Constraint File	Location
lab2_segment_scan.xdc	C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_08_segment_scan\constraints

☐ Copy constraints files into project

Add Sources → Add or create constraints → Next → Add Files → constraints/lab2_segment_scan.xdc. Copy constraints files into project 해제 → Finish. VS Code에서 편집한 원본을 계속 참조한다.

테스트벤치를 Top으로



Set as Top

Simulation Sources → sim_1을 펼친다. tb_segment_scan8 우클릭 → Set as Top. 보드용 lab2_segment_scan과 혼동하지 않는다.

서로 다른 Top 확인

The screenshot displays the IDE's project browser on the left and the settings panel on the right. The project browser shows a hierarchy where 'lab2_segment_scan' is under 'Design Sources' and 'tb_segment_scan8' is under 'sim_1' in 'Simulation Sources'. The settings panel shows the project name as 'lab2_segment_scan' and the top module name as 'lab2_segment_scan'.

Design Sources (1)

- lab2_segment_scan (lab2_segment_scan.v) (2)

Constraints (1)

Simulation Sources (2)

- sim_1 (2) (active)
 - tb_segment_scan8 (tb_segment_scan8.sv) (1)

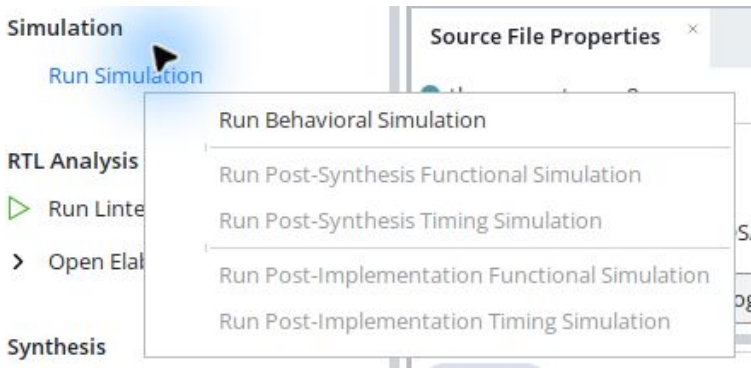
Hierarchy Libraries Compile Order

Settings Edit

Project name	lab2_segment_scan
Project location	C:/Users/UOS/Documents/e
Product family	Spartan-7
Project part	xc7s75fpga484-1
Top module name	lab2_segment_scan

Design Sources의 lab2_segment_scan과 Simulation Sources → sim_1의 tb_segment_scan8이 각각 굵게 표시되는지 확인한다.

XSim 실행



왼쪽 Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration이 끝날 때까지 기다린다.

1000 ns에서 끝내지 않기

⏮	Restart...	Ctrl+Shift+F5
▶	Run All	F3
▶ _{ns}	Run For...	
⏸	Step	F8
	Break	F5
✕	Delete All Breakpoints	
↺	Relaunch Simulation	

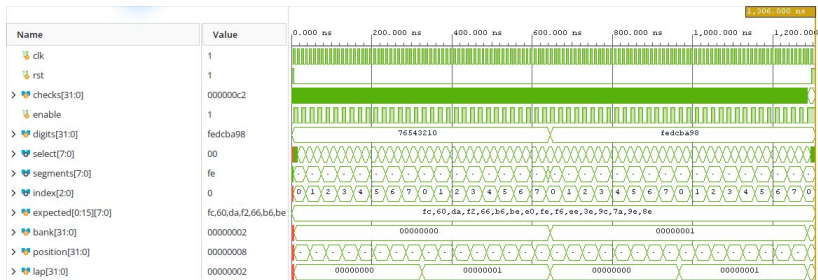
기본 실행은 1000 ns에서 멈춘다. 이때 checks=150으로 아직 검사가 끝나지 않았다. Run → Run All 또는 F3를 눌러 테스트벤치의 \$finish까지 실행한다.

XSim 194개 검사 통과

```
LAB2_PASS segment_scan8 checks=194
$finish called at time : 1306 ns :
.
```

Tcl Console에서 LAB2_PASS segment_scan8 checks=194와 \$finish 1306 ns를 확인한다. 오류로 중단되거나 검사 수가 다르면 통과한 것으로 기록하지 않는다.

전체 파형 확인



파형 탭 더블클릭 → Zoom Fit. digits는 76543210에서 fedcba98로 바뀐다. 각 입력 묶음에서 index=0~7을 두 번 순회한다. enable=0에서 유지되고 마지막 rst에서 index=0·select=00이 되는지 VS Code 파형과 대조한다.

핀 확인 화면 열기

Tcl Console	Messages	Log	Reports	Design Runs	Timing	Package Pins	I/O Ports	
Q	X	↕	메뉴	+	☰			
Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	
▽ All ports (35)								
> led (8)	OUT			✓	35	LVC MOS33* ▾	3.300	
> seg_com (8)	OUT			✓	(Multiple)	LVC MOS33* ▾	3.300	
> seg_data (8)	OUT			✓	(Multiple)	LVC MOS33* ▾	3.300	
> sw (8)	IN			✓	34	LVC MOS33* ▾	3.300	
▽ Scalar ports (3)								

Open Elaborated Design → OK. 오른쪽 위 레이아웃에서 I/O Planning 선택 → I/O Ports 탭 더블클릭. All ports는 35개다.

LED 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
v All ports (35)						
led (8)	OUT			✓	35	LVC MOS33* v
led[7]	OUT		L4 v	✓	35	LVC MOS33* v
led[6]	OUT		M4 v	✓	35	LVC MOS33* v
led[5]	OUT		M2 v	✓	35	LVC MOS33* v
led[4]	OUT		N7 v	✓	35	LVC MOS33* v
led[3]	OUT		M7 v	✓	35	LVC MOS33* v
led[2]	OUT		M3 v	✓	35	LVC MOS33* v
led[1]	OUT		M1 v	✓	35	LVC MOS33* v
led[0]	OUT		N5 v	✓	35	LVC MOS33* v

led 왼쪽 화살표를 펼쳐 8개 Package Pin을 XDC와 대조한다. led[0]부터 N5·M1·M3·M7·N7·M2·M4·L4다. 모든 I/O Std는 LVC MOS33이다.

자리 선택 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
v All ports (35)						
> led (8)	OUT			✓	35	LVC MOS33* v
v seg_com (8)	OUT			✓	(Multiple)	LVC MOS33* v
✓ seg_com[7]	OUT		H4	✓	35	LVC MOS33* v
✓ seg_com[6]	OUT		H6	✓	35	LVC MOS33* v
✓ seg_com[5]	OUT		G1	✓	36	LVC MOS33* v
✓ seg_com[4]	OUT		G3	✓	36	LVC MOS33* v
✓ seg_com[3]	OUT		L6	✓	35	LVC MOS33* v
✓ seg_com[2]	OUT		K1	✓	35	LVC MOS33* v
✓ seg_com[1]	OUT		K3	✓	35	LVC MOS33* v
✓ seg_com[0]	OUT		K5	✓	35	LVC MOS33* v

led를 접고 seg_com을 펼친다. seg_com[0]부터 K5·K3·K1·L6·G3·G1·H6·H4다. 코어 index=0은 비트 반전과 순서 변경을 거쳐 COM[7]을 선택한다.

세그먼트 데이터 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
v All ports (35)						
> led (8)	OUT			✓	35	LVC MOS33* v
> seg_com (8)	OUT			✓	(Multiple)	LVC MOS33* v
> seg_data (8)	OUT			✓	(Multiple)	LVC MOS33* v
seg_data[7]	OUT		F1	✓	36	LVC MOS33* v
seg_data[6]	OUT		F5	✓	36	LVC MOS33* v
seg_data[5]	OUT		E2	✓	36	LVC MOS33* v
seg_data[4]	OUT		E4	✓	36	LVC MOS33* v
seg_data[3]	OUT		J1	✓	35	LVC MOS33* v
seg_data[2]	OUT		J3	✓	35	LVC MOS33* v
seg_data[1]	OUT		J7	✓	35	LVC MOS33* v
seg_data[0]	OUT		H2	✓	35	LVC MOS33* v

seg_com을 접고 seg_data를 펼친다. seg_data[0]부터 H2·J7·J3·J1·E4·E2·F5·F1이다. 이름·비트 번호·Package Pin을 모두 확인한다.

스위치 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
v All ports (35)						
> led (8)	OUT			✓	35	LVC MOS33* ▾
> seg_com (8)	OUT			✓	(Multiple)	LVC MOS33* ▾
> seg_data (8)	OUT			✓	(Multiple)	LVC MOS33* ▾
sw (8)	IN			✓	34	LVC MOS33* ▾
sw[7]	IN		Y1	✓	34	LVC MOS33* ▾
sw[6]	IN		W3	✓	34	LVC MOS33* ▾
sw[5]	IN		U2	✓	34	LVC MOS33* ▾
sw[4]	IN		T1	✓	34	LVC MOS33* ▾
sw[3]	IN		W4	✓	34	LVC MOS33* ▾
sw[2]	IN		W1	✓	34	LVC MOS33* ▾
sw[1]	IN		V4	✓	34	LVC MOS33* ▾
sw[0]	IN		U4	✓	34	LVC MOS33* ▾

seg_data를 접고 sw를 펼친다. sw[0]부터 U4·V4·W1·W4·T1·U2·W3·Y1 이다. SW1~4는 sw[7:4]로 첫 자리 숫자를 정한다.

버튼·클록·리셋 핀 확인

Scalar ports (3)							
button	IN		N8	▼	✓	35	LVC MOS33* ▼
clk	IN		B6	▼	✓	36	LVC MOS33* ▼
rst	IN		K4	▼	✓	35	LVC MOS33* ▼

Scalar ports를 펼친다. button=N8, clk=B6, rst=K4다. B6 주 클록은 1 kHz다. 이 회로는 자동 스캔하므로 N8 버튼을 누를 필요가 없다.

합성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

왼쪽 Run Synthesis. Launch runs on local host와 jobs를 확인하고 OK.
이 실행은 jobs=4다.

합성 완료

 Synthesis successfully completed.

Next

- ☒ Run Implementation
- ☐ Open Synthesized Design
- ☐ View Reports

Synthesis successfully completed 확인 → Run Implementation 선택 → OK.

구현 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host와 jobs를 확인하고 OK. 구현 도중에는 RTL·XDC를 수정하지 않는다.

구현 완료

 Implementation successfully completed.

Next

☐ Open Implemented Design

☒ Generate Bitstream

☐ View Reports

Implementation successfully completed 확인 → Generate Bitstream 선택
→ OK.

비트스트림 생성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host와 jobs를 확인하고 OK. write_bitstream 완료까지 기다린다.

비트스트림 생성 완료

 Bitstream Generation successfully completed.

Next

- ☒ Open Implemented Design
- ☐ View Reports
- ☐ Open Hardware Manager
- ☐ Generate Memory Configuration File

Bitstream Generation successfully completed 확인. Open Implemented Design → OK. 이전 Elaborated Design을 닫을지 물으면 Yes.

타이밍 확인

Design Timing Summary

Setup		Hold	
Worst Negative Slack (WNS)	999998.062 ns	Worst Hold Slack (WHS)	0.119 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	17	Total Number of Endpoints	17

Timing → Design Timing Summary. Setup·Hold의 slack과 Number of Failing Endpoints를 확인한다.

검사 결과와 생성 파일

XSim: LAB2_PASS segment_scan8 checks=194, 1306 ns 종료. Icarus와 같은 TB와 원복한 RTL을 사용했다.

생성 파일:

`vivado/lab2_segment_scan.runs/impl_1/lab2_segment_scan.bit`.

GUI 합성·구현·bit 생성까지 확인했다. 장치 기록과 보드 사진·영상은 학생이 실습 장비에서 수행한다.

타이밍과 경고 해석

WNS=999998.062 ns, WHS=0.119 ns. Setup·Hold 실패 endpoint는 각각 0이다. 주 클럭 제약은 1,000,000 ns다.

TIMING-18: LED·자리 선택·세그먼트 데이터의 출력 24개에 외부 출력 지연이 지정되지 않았다. 비동기 입력에는 false path를 적용했다. 내부 타이밍 통과가 외부 입출력 타이밍까지 보장하지는 않는다.

CFGBVS-1: 설정 बैं크 전압 속성이 없다. 보드 회로도와 설정 전압을 확인하여 값을 정한다. 경고만 없애려고 임의의 전압을 입력하지 않는다.

장치에서 동작 확인

1. 보드 전원·JTAG·주 클록 1 kHz 설정을 확인한다.
2. Open Hardware Manager → Open target → Auto Connect.
3. FPGA를 우클릭 → Program Device. 방금 생성한 lab2_segment_scan.bit를 선택하고 Program.
4. 완료 상태를 확인하고 K4로 초기화한다.
5. SW1~4로 숫자를 바꿔 표시 결과를 예상과 비교한다.

장비에서 확인할 순서

1. 주 클럭을 1 kHz로 맞추고 K4로 초기화한다.
2. SW1..4를 1001로 정한다. COM[7]부터 9,1,2,3,4,5,6,7을 확인한다.
3. SW1..4를 바꿔 첫 자리만 변하는지 확인한다. N8은 스캔 진행에 필요하지 않다.
4. LED[2:0]은 현재 index다. 빠른 변화를 눈으로 모두 구분할 수 있다고 가정하지 않는다.
5. 카메라 줄무늬나 일부 자리 누락은 촬영 주기와 스캔 주기의 차이도 확인한다. 정상 숫자 사진과 입력 변경 영상을 함께 남긴다.

실험 후 레포트와 GitHub

1. 도구 버전·source commit·XDC·bit 해시·타이밍과 경고를 적는다.
2. Vivado 시뮬레이션 파형·검사 결과와 장치 기록 화면을 넣는다.
3. 보드 전체 사진과 스위치 변경에 따른 숫자 변화 영상을 찍는다.
4. 소스·실험 전후 레포트·사진·영상 링크를 GitHub에서 연결한다.